COS202 PROGRAMMING II – LAB ASSESSMENT PROJECT

Case Study: Smart Clinic Appointment & Patient Management System (SCAPMS)

PROJECT OVERVIEW

A university clinic wants to develop a **Smart Clinic Appointment & Patient Management System (SCAPMS)** to manage patients, medical services, appointments, queues, consultations, and basic clinic reports.

The system should provide an interactive graphical user interface through which clinic staff can register patients, schedule and manage appointments, search and sort patient records, manage waiting queues, and generate simple reports. You are required to build a **fully functional C++ application** with the following requirements.

CORE REQUIREMENTS

1. Advanced Object-Oriented Programming

Design a class hierarchy containing:

a. Abstract Class

Create an abstract class called **PatientRecord** which should contain common information such as:

i. Patient ID
ii. Name
iii. Age
iv. Contact information

This class must contain at least one **pure virtual function**.

b. Subclasses

Create at least three subclasses:

i. GeneralPatient

ii. EmergencyPatient

iii. FollowUpPatient

Each subclass should provide behaviour appropriate to its patient type.

c. Interface / Abstract Interface

Create an interface-like abstract class called **Appointable** which should define operations that can be implemented by patient types that can be scheduled for appointments. For example:

i. scheduleAppointment()

ii. cancelAppointment()

d. Polymorphism

Implement a function that can process different PatientRecord objects through a common base-class pointer or reference. (The program should demonstrate **runtime polymorphism**.)

e. Encapsulation and Composition

-Create a class called **ClinicDatabase** which contains a collection of patient records.

-Also create an **Appointment** class containing relevant appointment information. A patient should be associated with one or more appointments.

f. Class Organisation

You must organise their C++ source code using appropriate **header/source files and namespaces**.

For example, you may use a structure as follows:

/model

PatientRecord.h

PatientRecord.cpp

GeneralPatient.h

GeneralPatient.cpp

EmergencyPatient.h

EmergencyPatient.cpp

FollowUpPatient.h

FollowUpPatient.cpp

Appointment.h

Appointment.cpp

/controller

ClinicManager.h

ClinicManager.cpp

SearchEngine.h

SearchEngine.cpp

AppointmentController.h

AppointmentController.cpp

/gui

MainWindow.h

MainWindow.cpp

PatientPanel.h

PatientPanel.cpp

AppointmentPanel.h

AppointmentPanel.cpp

/utils

IDGenerator.h

FileHandler.h

Note: The exact organisation may differ provided the program is logically structured.

2. Class Hierarchies & Program Organisation

The application must demonstrate clear separation of responsibilities. You are required to organise the application into logical components such as:

Model

- PatientRecord
- GeneralPatient
- EmergencyPatient
- FollowUpPatient
- Appointment

Controller

- ClinicManager
- SearchEngine
- AppointmentController

GUI

- MainWindow
- PatientPanel
- AppointmentPanel
- AdminPanel
- SearchPanel

Utilities

- IDGenerator
- FileHandler

Use **C++ namespaces** to avoid naming conflicts and improve program organisation. Your design should demonstrate that the GUI, business logic, and data-handling components are not unnecessarily combined into one large class.

3. Using Data Structures

The system must use appropriate C++ data structures as follows:

a. Vector

Use ***std::vector*** to store registered patients.

b. Queue

Use ***std::queue*** to manage patients waiting for consultation. (The patient who joins the queue first should normally be served first.)

c. Stack

Use ***std::stack*** to implement an **Undo Last Administrative Operation** feature.

For example, an administrator may:

i. Delete a patient

ii. Remove an appointment

iii. Modify a patient record

The system should allow the most recent administrative operation to be reversed.

d. Array

Use a fixed-size C++ array to implement a small cache for **"Frequently Accessed Patient Records".**

For example:

PatientRecord* recentPatients[10];

[The implementation may use a different appropriate design.]

4. Searching Algorithms

You must implement at least **two searching algorithms manually**.

a. Linear Search to search for a patient by **Patient ID.**

b. Binary Search: Implement binary search for a suitable field after the patient records have been sorted.

c. Recursive Search

You may also implement a recursive search for **Patient name.**

The program should use an appropriate searching technique depending on the state of the data. [eg. Sorted or unsorted].

5. Sorting Algorithms

Implement **two sorting algorithms manually**.

You may choose from:

i. Selection Sort
ii. Insertion Sort
iii. Merge Sort
iv. Quick Sort

The system should allow patient records to be sorted by at least two of the following:

i. Patient name
ii. Patient ID
iii. Age
iv. Appointment date

Algorithm Selection

The GUI should provide a dropdown/combo box allowing the user to select the sorting algorithm.

For example:

Sort Algorithm:

[Selection Sort ▼]

Sort By:

[Patient Name ▼]

[SORT]

You must explain why the selected sorting algorithm is appropriate for the particular situation.

6. Event-Driven Programming

The GUI application must respond to events generated by the user or system. Examples include:

a. Button Events

Examples:

i. Add Patient
ii. Delete Patient
iii. Schedule Appointment
iv. Cancel Appointment
v. Search
vi. Sort
vii. Save
viii. Load

b. Menu Events

For example:

File

├── New

├── Save

├── Load

└── Exit

c. Text Field Events

The system may respond when the user enters or changes:

i. Patient ID

ii. Patient name

iii. Search keywords

iv. Appointment information

d. Timer Events

The system may use a timer to:

i. Display appointment reminders

ii. Alert staff about waiting patients

iii. Refresh dashboard statistics

You must use appropriate **event handlers/listeners/signals and slots**, depending on the C++ GUI framework selected.

7. GUI Programming, Components & Layout Management

You must develop a graphical user interface for the system. The main window should contain multiple sections or tabs such as:

a. Patient Records

Used to:

i. View patients

ii. Add patients

iii. Edit patients

iv. Delete patients

b. Appointments

Used to:

i. Schedule appointments

ii. Cancel appointments

iii. View appointments

c. Waiting Queue

Used to:

i. View waiting patients

ii. Add patients to queue

iii. Call the next patient

d. Search & Sort

Used to:

i. Search patient records

ii. Select search algorithm

iii. Select sorting algorithm

iv. Sort records

The GUI should contain appropriate:

i. Tables/list views

ii. Buttons

iii. Text fields

iv. Combo boxes

v. Labels

vi. Dialog boxes

vii. Menus

viii. Status information

8. Layout Management

You must use appropriate GUI layout management rather than relying entirely on fixed/manual positioning. The application should demonstrate at least **two appropriate layout techniques** available in the selected C++ GUI framework.

For example, when using Qt:

i. QVBoxLayout

ii. QHBoxLayout

iii. QGridLayout

iv. QFormLayout

v. QStackedLayout

You should explain why the selected layouts are appropriate.

9. Advanced GUI Techniques

Students must implement at least **two** of the following:

a. Custom Table/List Presentation

Use custom formatting or rendering to visually distinguish patient categories or appointment statuses. For example:

Emergency → High Priority

General → Normal

Follow-Up → Scheduled

b. Dynamic Components

Allow the interface to dynamically add or remove fields depending on the selected patient type.

c. File Dialog

Provide a file chooser for importing or exporting patient information.

d. Timers

Use timers for:

i. Appointment reminders

ii. Dashboard refresh

iii. Waiting-time notifications

e. Input Validation

Display appropriate dialog messages when invalid information is entered.

Examples:

Error: Patient age must be between 0 and 120.

Error: Patient ID already exists.

f. Keyboard Shortcuts

Implement useful shortcuts such as:

Ctrl + S → Save

Ctrl + F → Search

Ctrl + Q → Exit

g. Tooltips

Provide helpful tooltips for important buttons and controls.

10. Exception Handling

The application must demonstrate appropriate exception handling.

Students should use:

```
try
{
  // operation
}
catch (...)
{
  // handle error
}
```

The system should handle situations such as:

i. Invalid patient information
ii. Duplicate patient ID
iii. Invalid age
iv. Invalid appointment data
v. File loading failure
vi. Invalid search input

You are encouraged to create at least one **custom exception class** where appropriate. The program should display meaningful error messages instead of terminating unexpectedly.

11. GUI Application & Program Development

The final system must be:

a. Interactive

Users should be able to perform the major operations through the GUI.

b. Robust

The system should continue operating appropriately after normal user errors.

c. Error-Tolerant

Invalid operations should be handled using appropriate exception handling and validation.

d. Persistent

Patient and appointment data should be saved and loaded using text or structured data files.

You may use formats such as:

.txt

.csv

.json

MANDATORY SYSTEM FEATURES

The completed system must provide the following features:

✔ Register new patients

✔ Edit patient information

✔ Delete patient records

✔ Schedule appointments

✔ Cancel appointments

✔ Maintain a waiting queue

✔ Search patients by name, ID, or category

✔ Sort patient records using selectable algorithms

✔ Demonstrate at least two student-implemented sorting algorithms

✔ Demonstrate at least two student-implemented searching algorithms

✔ Demonstrate a recursive algorithm

✔ Implement an Undo Last Administrative Operation feature using a Stack

✔ Display a GUI dashboard with multiple panels

✔ Save and load patient/appointment data from files

✔ Handle invalid input using validation and exception handling

✔ Generate simple reports

SIMPLE REPORTS

The system should generate at least three reports, such as:

1. Most Frequently Attended Patients

Display patients with the highest number of completed appointments.

2. Patients with Pending Appointments

Display patients who have appointments that have not yet been completed.

3. Patient Category Distribution

For example:

General Patients 42

Emergency Patients 8

Follow-Up Patients 25

4. Daily Appointment Summary

For example:

Total Appointments: 35

Completed: 20

Pending: 10

Cancelled: 5

Students may implement additional useful reports.

SUBMISSION REQUIREMENTS

You must submit the following a single pdf file/document:

1. Source Code

The complete C++ source code appropriately labelled with their headings such as:

i. Header files
ii. Source files
iii. GUI files/resources
iv. Data files required to run the application

2. Screenshots

Screenshots showing the major parts of the GUI, including:

i. Main dashboard
ii. Patient registration
iii. Appointment management
iv. Search/sort interface
v. Waiting queue
vi. Reports

3. UML/Class Diagram

A diagram showing the major classes and their relationships.

The diagram should clearly demonstrate:

i. Inheritance

ii. Abstract classes

iii. Polymorphism

iv. Composition/association

4. Project Report

Submit a **2–3 page report** containing:

i. Project description

ii. System features

iii. Class hierarchy

iv. Data structures used

v. Searching algorithms used

vi. Sorting algorithms used

vii. Recursive algorithm used

viii. GUI design

ix. Exception handling approach

x. Algorithms chosen and reasons

xi. Challenges encountered

xii. How the challenges were resolved

WARNING: IMPORTANT IMPLEMENTATION REQUIREMENT

You are expected to **implement the major algorithms yourself**. Do NOT simply replace the required implementations with built-in library functions. In particular, the following must be student-implemented:

i. Searching algorithms

ii. Sorting algorithms

iii. Recursive component

Standard C++ containers such as:

std::vector

std::queue

std::stack

may be used for data storage and management, but the required searching and sorting algorithms must be written by you.